

P1111R0

Resolutions to NB Comments on the Parallelism TS v2

Published Proposal, 4 June 2018

This version:

<http://wg21.link/D1111R0>

Issue Tracking:

[GitHub](#)

Authors:

[Bryce Adelstein Lelbach](#) (NVIDIA)

[Matthias Kretz](#)

Audience:

SG1, LEWG, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1 CA 15, CA 16, CA 17, US 11, US 12, CH 35

2 US 37

3 US 44

4 US 53

5 US 3

6 US 7

7 CA 4

8 CA 8

9 CA 5

10 DE 19, DE 48

11 CH 21

12 CH 23

13 US 42, US 43

14 CH 30, CH 33

15 CH 38

16 US 41, US 45, CH 46, CH 47

17 US 50

18 US 51

References

Informative References

NOTE: All wording is relative to [\[N4744\]](#), the Proposed Draft Technical Specification for version 2 of the C++ Parallelism Technical Specification (ISO/IEC PTDS 19750).

NOTE: Paragraph references are relative to [\[N4744\]](#) and do not take paragraph renumbering into account. In the places where this would be ambiguous, descriptions of the location are provided instead of paragraph numbers.

NOTE: The changes should be applied in order.

§ 1. CA 15, CA 16, CA 17, US 11, US 12, CH 35

Modify **[parallel.simd.whereexpr]** paragraph 12 as follows:

If the template parameter `Flags` is `vector_aligned_tag`, `mem` shall point to storage aligned by `memory_alignment_v<T, U>`. If the template parameter `Flags` is `overaligned_tag<N>`, `mem` shall point to storage aligned by `N`. If the template parameter `Flags` is `element_aligned_tag`, `mem` shall point to storage aligned by `alignof(U)`. If `M` is not `bool`, the largest ~~$i \in \{0, M::size() - 1\}$~~ **selected index** where `mask[i]` is `true` is less than the number of values pointed to by `mem`.

In **[parallel.simd.reductions]** replace all occurrences of:

for all ~~$i \in \{j \in \mathbb{N}_0 \mid j < M::size() \wedge \text{mask}[j]\}$~~

with:

for all **selected indices i**

Replace all occurrences of ~~$\in$~~ preceding a half-open range with **in the range of**.

Replace all remaining occurrences of ~~$\in$~~ with **$\in$** .

Replace all occurrences of ~~$\mathbb{N}_0$~~ with **$\mathbb{N}$** .

Replace all occurrences of ~~$-th$~~ to **th** .

Replace all occurrences of `i` and `j` (code font) with i and j (math font) respectively.

§ 2. US 37

Move paragraphs 4 and 5 of `[parallel.simd.overview]` into a new subsection after `[parallel.simd.overview]`:

9.3.2 simd width `[parallel.simd.width]`

```
static constexpr size_t size() noexcept;
```

Returns: The width of `simd<T, Abi>`.

Move paragraphs 4 and 5 of `[parallel.simd.mask.overview]` into a new subsection after `[parallel.simd.mask.overview]`:

9.5.2 simd_mask width `[parallel.simd.mask.width]`

```
static constexpr size_t size() noexcept;
```

Returns: The width of `simd<T, Abi>`.

§ 3. US 44

Modify `[parallel.references]` paragraph 2 as follows:

ISO/IEC 14882:2017 is herein called the **C++ Standard**. *References to clauses within the C++ Standard are written as "C++17 §3.2"*. The library described in ISO/IEC 14882:2017 clauses 20-33 is herein called the **C++ Standard Library**. The C++ Standard Library components described in ISO/IEC 14882:2017 clauses 28, 29.8 and 23.10.10 are herein called the **C++ Standard Algorithms Library**.

Replace all references to the C++ Standard with references in the style "C++17 §3.2":

Modify `[parallel.simd.ctor]` paragraph 7 as follows:

if both `U` and `value_type` are integral, the integer conversion rank `[conv.rank]` (C++17 §7.15) of `value_type` is greater than the integer conversion rank of `U`.

Modify **[parallel.simd.ctor]** paragraph 11 as follows:

Vectorization-unsafe standard library functions may not be invoked by gen
(~~algorithms.parallel.exec~~C++17 §28.4.3).

Modify **[parallel.simd.reductions]** paragraph 4 as follows:

```
template<class T, class Abi, class BinaryOperation = plus<>>
T reduce(const simd<T, Abi>& x, BinaryOperation binary_op = {});
```

Requires: `binary_op` shall be callable with two arguments of type `T` returning `T`, or callable with two arguments of type `simd<T, A1>` returning `simd<T, A1>` for every `A1` that is an ABI tag type.

Returns: `GENERALIZED_SUM(binary_op, x.data[i], ...)` for all $i \in [0, \text{size}())$
(C++17 §29.2).

§ 4. US 53

Modify **[parallel.scope]** paragraph 3 as follows:

The goal of this Technical Specification is to build widespread existing practice for parallelism in the C++ ~~programming language~~~~standard algorithms library~~. It gives advice on extensions to those vendors who wish to provide them.

§ 5. US 3

Insert a new paragraph after **[parallel.general.namespaces]** p1:

Each header described in this technical specification shall import the contents of `std::experimental::parallelism_v2` into `std::experimental` as if by

```
namespace std::experimental {
    inline namespace parallelism_v2 {}
}
```

§ 6. US 7

Modify [**parallel.alg.ops.synopsis**] as follows:

```
// Exposition only: Suppress template argument deduction.  
template<class T> struct no_deducetype_identity { using type = T; };  
template<class T> using no_deduce_ttype_identity_t = typename  
no_deducetype_identity<T>::type;
```

In [**parallel.alg**], replace all occurrences of:

```
no_deduce_t
```

with:

```
type_identity_t
```

In [**parallel.simd**], replace all occurrences of:

```
no_deduce_t
```

with:

```
type_identity_t
```

§ 7. CA 4

Remove the column titled "Doc. No." from [**parallel.general.features**] Table 1.

§ 8. CA 8

Modify [**parallel.alg.reductions**] paragraph 3 as follows:

Modifications to the accumulator by the application of element access functions accrue as partial results. At some point before the algorithm returns, the partial results are combined, two at a time, using the reduction object's combiner operation until a single value remains, which is then assigned back to the live-out object. [*Note:* ~~in~~In order to produce useful results, modifications to the accumulator should be limited to commutative operations closely related to the combiner operation. For example if the combiner is `plus<T>`, incrementing the accumulator would be consistent with the combiner but doubling it or assigning to it would not. — *end note*]

§ 9. CA 5

Modify `[parallel.exceptions.synopsis]` p2 as follows:

~~The type `exception_list::iterator` fulfills the requirements of `ForwardIterator`.~~
`exception_list::iterator` is an iterator which meets the forward iterator requirements and has a value type of `exception_ptr`.

§ 10. DE 19, DE 48

Modify the array-returning declarations of `split` in `[parallel.simd.synopsis]`:

```
template<size_t... Sizes, class T, class Abi>
    tuple<simd<T, simd_abi::deduce_t<T, Sizes>>...>
        split(const simd<T, Abi>&);
template<size_t... Sizes, class T, class Abi>
    tuple<simd_mask<T, simd_mask_abi::deduce_t<T, Sizes>>...>
        split(const simd_mask<T, Abi>&);
template<class V, class Abi>
    array<V, simd_size_v<typename V::value_type, Abi> / V::size()>
        split(const simd<typename V::value_type, Abi>&);
template<class V, class Abi>
    array<V, simd_size_v<typename V::simd_type::value_type, Abi> / V::size()>
        split(const simd_mask<typename V::simd_type::value_type, Abi>&);
```

Modify the array-returning definitions of `split` in `[parallel.simd.casts]`:

```

template<class V, class Abi>
    array<V, simd_size_v<typename V::value_type, Abi> / V::size()>
        split(const simd<typename V::value_type, Abi>& x);
template<class V, class Abi>
    array<V, simd_size_v<typename V::simd_type::value_type, Abi> / V::size()>
        split(const simd_mask<typename V::simd_type::value_type, Abi>& x);

```

Returns: An array of data-parallel objects with the i -th `simd/simd_mask` element of the j -th element initialized to the value of the element in `x` with index $i + j * V::size()$.

Remarks: These functions shall not participate in overload resolution unless **either**:

- ~~`simd_size_v<typename V::value_type, Abi>` is an integral multiple of `V::size()`, and~~
- ~~for the overload with a `simd` parameter `is_simd_v<V>` is true, for the overload with a `simd_mask` parameter `is_simd_mask_v<V>` is true.~~
- `is_simd_v<V>` is true and `simd_size_v<typename V::value_type, Abi>` is an integral multiple of `V::size()`, or
- `is_simd_mask_v<V>` is true and `simd_size_v<typename V::simd_type::value_type, Abi>` is an integral multiple of `V::size()`.

§ 11. CH 21

Modify the synopsis for namespace `simd_abi` in `[parallel.simd.abi]` as follows:

```

namespace simd_abi {
    struct scalar {};
    template<int N> struct fixed_size {};
    template<class T> inline constexpr int max_fixed_size = implementation-defined;
    template<class T> using compatible = implementation-defined;
    template<class T> using native = implementation-defined;
}

```

§ 12. CH 23

Modify `[parallel.simd.abi]` paragraph 9 as follows:

`compatible<T>` is an implementation-defined alias for an ABI tag. [*Note:* The intent is to use the ABI tag producing the most efficient data-parallel execution for the element type `T` that ensures ABI compatibility between translation units on the target architecture. — *end note*]

[*Example:* Consider a target architecture supporting the extended ABI tags `__simd128` and `__simd256`, where the `__simd256` type requires an optional ISA extension on said architecture. Also, the target architecture does not support long double with either ABI tag. The implementation therefore defines `compatible<T>` as an alias for:

- ~~`compatible<T>` is an alias for `__simd128` for all vectorizable `T`, except `long double`, and~~
- ~~`compatible<long double>` as an alias for scalar.~~
- `scalar` if `T` is the same type as `long double`, and
- `__simd128` otherwise. — *end example*]

§ 13. US 42, US 43

Modify [`parallel.simd.binary`] as follows:

```
friend simd operator+(const simd& lhs, const simd& rhs);
friend simd operator-(const simd& lhs, const simd& rhs);
friend simd operator*(const simd& lhs, const simd& rhs);
friend simd operator/(const simd& lhs, const simd& rhs);
friend simd operator%(const simd& lhs, const simd& rhs);
friend simd operator&(const simd& lhs, const simd& rhs);
friend simd operator|(const simd& lhs, const simd& rhs);
friend simd operator^(const simd& lhs, const simd& rhs);
friend simd operator<<(const simd& lhs, const simd& rhs);
friend simd operator>>(const simd& lhs, const simd& rhs);
```

Returns: A `simd` object initialized with the results of ~~the element-wise application of the indicated operator~~ applying the indicated operator to `lhs` and `rhs` as a binary element-wise operation.

Throws: Nothing.

Remarks: Each of these operators shall not participate in overload resolution unless the indicated operator can be applied to objects of type `value_type`.

Modify [`parallel.simd.cassign`] as follows:

```

friend simd& operator+=(simd& lhs, const simd& rhs);
friend simd& operator-=(simd& lhs, const simd& rhs);
friend simd& operator*=(simd& lhs, const simd& rhs);
friend simd& operator/=(simd& lhs, const simd& rhs);
friend simd& operator%=(simd& lhs, const simd& rhs);
friend simd& operator&=(simd& lhs, const simd& rhs);
friend simd& operator|=(simd& lhs, const simd& rhs);
friend simd& operator^=(simd& lhs, const simd& rhs);
friend simd& operator<=(simd& lhs, const simd& rhs);
friend simd& operator>=(simd& lhs, const simd& rhs);
friend simd& operator<<=(simd& lhs, int n);
friend simd& operator>>=(simd& lhs, int n);

```

Effects: These operators ~~perform the indicated binary element-wise operation.~~ apply the indicated operator to lhs and rhs as an element-wise operation.

Returns: lhs.

Throws: Nothing.

Remarks: These operators shall not participate in overload resolution unless the indicated operator can be applied to objects of type `value_type`.

```

friend simd& operator<<=(simd& lhs, int n);
friend simd& operator>>=(simd& lhs, int n);

```

Effects: Equivalent to: `return operator@=(lhs, simd(n));`

Remarks: These operators shall not participate in overload resolution unless the indicated operator can be applied to objects of type `value_type`.

Modify `[parallel.simd.comparison]` as follows:

```
friend mask_type operator==(const simd& lhs, const simd& rhs);
friend mask_type operator!=(const simd& lhs, const simd& rhs);
friend mask_type operator>=(const simd& lhs, const simd& rhs);
friend mask_type operator<=(const simd& lhs, const simd& rhs);
friend mask_type operator>(const simd& lhs, const simd& rhs);
friend mask_type operator<(const simd& lhs, const simd& rhs);
```

Returns: A simd_mask object initialized with the results of ~~the element-wise application of the indicated operator.~~ applying the indicated operator to lhs and rhs as a binary element-wise operation.

Throws: Nothing.

Modify [parallel.simd.mask.binary] as follows:

```
friend simd_mask operator&&(const simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask operator||(const simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask operator& (const simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask operator| (const simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask operator^ (const simd_mask& lhs, const simd_mask& rhs) noexcept;
```

Returns: A simd_mask object initialized with the results of ~~the element-wise application of the indicated operator.~~ applying the indicated operator to lhs and rhs as a binary element-wise operation.

Modify [parallel.simd.mask.cassign] as follows:

```
friend simd_mask& operator&=(simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask& operator|=(simd_mask& lhs, const simd_mask& rhs) noexcept;
friend simd_mask& operator^=(simd_mask& lhs, const simd_mask& rhs) noexcept;
```

Effects: These operators ~~perform the indicated binary element-wise operation.~~ apply the indicated operator to lhs and rhs as a binary element-wise operation.

Returns: lhs.

Modify [parallel.simd.mask.comparison] as follows:

```
friend simd_mask operator==(const simd_mask& lhs, const simd_mask& rhs) noexcept;  
friend simd_mask operator!=(const simd_mask& lhs, const simd_mask& rhs) noexcept;
```

Returns: An object initialized with the results of ~~the element-wise application of the indicated operator~~ applying the indicated operator to lhs and rhs as a binary element-wise operation.

§ 14. CH 30, CH 33

Add noexcept to the following functions in **[parallel.simd]**:

- simd_cast
- static_simd_cast
- split
- concat
- reduce(const const_where_expression<M, V>& x, plus<> binary_op)
- reduce(const const_where_expression<M, V>& x, multiplies<> binary_op)
- reduce(const const_where_expression<M, V>& x, bit_and<> binary_op)
- reduce(const const_where_expression<M, V>& x, bit_or<> binary_op)
- reduce(const const_where_expression<M, V>& x, bit_xor<> binary_op)
- hmin
- hmax
- All members of const_where_expression and where_expression
- All simd constructors, except simd(const U* mem, Flags f)
- All simd member and non-member operators, except operator[]
- All reference member functions
- simd_mask()

Remove noexcept from the following functions in **[parallel.simd]**:

- find_first_set (only the non-simd overload)
- find_last_set (only the non-simd overload)
- clamp (only in the synopsis)

Remove the following *Throws*: paragraphs:

- `[parallel.simd.whereexpr]` paragraph 10
- `[parallel.simd.ctor]` paragraph 3
- `[parallel.simd.unary]` paragraph 5, paragraph 9, paragraph 13, paragraph 17, paragraph 20, paragraph 23, paragraph 27, and paragraph 30
- `[parallel.simd.binary]` paragraph 3 and paragraph 7
- `[parallel.simd.cassign]` paragraph 4
- `[parallel.simd.comparison]` paragraph 3
- `[parallel.simd.reductions]` paragraph 12, paragraph 15, paragraph 19, paragraph 23, paragraph 26, paragraph 29, paragraph 32, paragraph 35
- `[parallel.simd.casts]` paragraph 4 and paragraph 10

Modify `[parallel.simd.overview]` paragraph 2 as follows:

Every specialization of `simd` shall be a complete type. The specialization `simd<T, Abi>` is supported if `T` is a vectorizable type and

- `Abi` is `simd_abi::scalar`, or
- `Abi` is `simd_abi::fixed_size<N>`, with `N` **is** constrained as defined in 9.2.1.

If `Abi` is an extended ABI tag, it is implementation-defined whether `simd<T, Abi>` is supported.

[*Note*: The intent is for implementations to decide on the basis of the currently targeted system. — *end note*]

If `simd<T, Abi>` is not supported, the specialization shall have a deleted default constructor, deleted destructor, deleted copy constructor, and deleted copy assignment. **Otherwise, the following are true:**

- `is_nothrow_move_constructible_v<simd<T, Abi>>`, and
- `is_nothrow_move_assignable_v<simd<T, Abi>>`, and
- `is_nothrow_default_constructible_v<simd<T, Abi>>`.

Modify `[parallel.simd.mask.overview]` paragraph 2 as follows:

Every specialization of `simd_mask` shall be a complete type. The specialization `simd_mask<T, Abi>` is supported if `T` is a vectorizable type and

- `Abi` is `simd_abi::scalar`, or
- `Abi` is `simd_abi::fixed_size<N>`, with `N` constrained as defined in (9.2.1).

If `Abi` is an extended ABI tag, it is implementation-defined whether `simd_mask<T, Abi>` is supported. [*Note*: The intent is for implementations to decide on the basis of the currently targeted system. — *end note*]

If `simd_mask<T, Abi>` is not supported, the specialization shall have a deleted default constructor, deleted destructor, deleted copy constructor, and deleted copy assignment. Otherwise, the following are true:

- `is_nothrow_move_constructible_v<simd_mask<T, Abi>>`, and
- `is_nothrow_move_assignable_v<simd_mask<T, Abi>>`, and
- `is_nothrow_default_constructible_v<simd_mask<T, Abi>>`.

After **[parallel.simd.mask.ctor]** paragraph 8 add a new paragraph:

Throws: Nothing.

After **[parallel.simd.mask.copy]** paragraph 3 and paragraph 7 add a new paragraph:

Throws: Nothing.

After **[parallel.simd.mask.reductions]** paragraph 13, paragraph 16, and paragraph 22 add a new paragraph:

Throws: Nothing.

§ 15. CH 38

Modify **[parallel.simd.overview]** as follows:

```
template<class T, class Abi> class simd {
public:
    using value_type = T;
    using reference = see below;
    using mask_type = simd_mask<T, Abi>;
    using abi_type = Abi;
```

§ 16. US 41, US 45, CH 46, CH 47

Modify the definition of `simd`'s U&& constructor in `[parallel.simd.ctor]` as follows:

```
template<class U> simd(U&&);
```

Effects: Constructs an object with each element initialized to the value of the argument after conversion to `value_type`.

Throws: Any exception thrown while converting the argument to `value_type`.

Remarks: Let `From` ~~denote~~ ~~identify~~ the type `remove_cv_t<remove_reference_t<U>>`. This constructor shall not participate in overload resolution unless:

- `From` is a vectorizable type and every possibly value of `From` can be represented with `value_type`, or
- `From` is not an arithmetic type and is implicitly convertible to `value_type`, or
- `From` is `int`, or
- `From` is `unsigned int` and `value_type` is an unsigned integral type.

Modify the definition of `simd_cast` in `[parallel.simd.casts]` as follows:

```
template<class T, class U, class Abi> see below simd_cast(const simd<U, Abi>
```

Let T_o ~~denote~~~~identify~~ $T::\text{value_type}$ if $\text{is_simd_v}<T>$ is true, or T otherwise.

Returns: A `simd` object with the i^{th} element initialized to `static_cast<To>(x[i])` for all $i \in [0, \text{size}())$.

Throws: Nothing.

Remarks: The function shall not participate in overload resolution unless

- every possible value of type U can be represented with type T_o , and
- either
 - $\text{is_simd_v}<T>$ is false, or
 - $T::\text{size}() == \text{simd}<U, \text{Abi}>::\text{size}()$ is true.

The return type is

- T if $\text{is_simd_v}<T>$ is true, ~~otherwise;~~
- ~~otherwise,~~ `simd<T, Abi>` ~~is if~~ U is the same type as T , ~~otherwise;~~
- ~~otherwise,~~ `simd<T, simd_abi::fixed_size<simd<U, Abi>::size()>>`

Modify the definition of `static_simd_cast` in [**parallel.simd.casts**] as follows:


```
template<class T, class U, class Abi> see below static_simd_cast(const sim
```

Let To ~~denote~~~~identify~~ $T::value_type$ if $is_simd_v<T>$ is true or T otherwise.

Returns: A `simd` object with the i^{th} element initialized to `static_cast<To>(x[i])` for all $i \in [0, size())$.

Throws: Nothing.

Remarks: The function shall not participate in overload resolution unless either:

- $is_simd_v<T>$ is false, or
- $T::size() == simd<U, Abi>::size()$ is true.

The return type is:

- T if $is_simd_v<T>$ is true, ~~otherwise;~~
- ~~otherwise,~~ `simd<T, Abi>` if either U is the same type as T or ~~U and T are integral types that only differ in signedness, otherwise~~ `make_signed_t<U>` is the same type as `make_signed_t<T>`;
- ~~otherwise,~~ `simd<T, simd_abi::fixed_size<simd<U, Abi>::size()>>`.

§ 17. US 50

Modify `[parallel.simd.math]` paragraph 2 as follows:

Each function overload produced by the above rules applies the indicated `<cmath>` function element-wise. ~~The results per element are not required to be bitwise~~ `For the mathematical functions, the results per element only need to be approximately` equal to the application of the function which is overloaded for the element type.

§ 18. US 51

Modify `[parallel.simd.synopsis]` as follows:

```
bool all_of (see belowT) noexcept;  
bool any_of (see belowT) noexcept;  
bool none_of (see belowT) noexcept;  
bool some_of (see belowT) noexcept;  
int popcount (see belowT) noexcept;  
int find_first_set(see belowT) noexcept;  
int find_last_set (see belowT) noexcept;
```

Modify [parallel.simd.mask.reductions] as follows:

```
bool all_of (see belowT) noexcept;  
bool any_of (see belowT) noexcept;  
bool none_of (see belowT) noexcept;  
bool some_of (see belowT) noexcept;  
int popcount (see belowT) noexcept;
```

Returns: all_of and any_of return their arguments; none_of returns the negation of its argument; some_of returns false; popcount returns the integral representation of its argument.

Remarks: ~~The functions shall not participate in overload resolution unless the argument is of type bool.~~ The parameter type *T* is an unspecified type that is only constructible via implicit conversion from bool.

```
int find_first_set(see belowT) noexcept;  
int find_last_set (see belowT) noexcept;
```

Requires: The value of the argument is true.

Returns: 0.

Remarks: ~~The functions shall not participate in overload resolution unless the argument is of type bool.~~ The parameter type *T* is an unspecified type that is only constructible via implicit conversion from bool.

§ References

§ Informative References

[N4744]

ISO/IEC JTC1/SC22/WG21. Programming Languages — Technical Specification for C++ Extensions for Parallelism Version 2. Proposed Draft Technical Specification. URL: <https://wg21.link/N4744>